

AXLY

DSA Tracker

PRD & Technical Documentation

Product Requirements • Architecture • Database Schema • API Plan

dsatracker.axly.in

Table of Contents

1. Executive Summary
2. Problem Statement
3. V1 Objectives
4. Scope
5. V1 Locked Decisions
6. User Roles
7. Core User Flow
8. Admin Flow
9. Functional Requirements
10. Technology Stack
11. System Architecture
12. Architecture Components
13. Authentication & Authorization

Implementation note: users.id must use the same UUID as auth.users.id from Supabase Auth. New users are created with role = user by default; only an authorized existing admin may promote a user to admin.

14. Entity Relationship Diagram
15. Database Table Specifications
16. Status Value Definitions
17. Daily Question Behavior
18. Assignment & Unassignment Behavior
19. Progress & Completion Model
20. API Design

21. Security Requirements

RLS implementation rule: backend operations must preserve the authenticated user's context whenever PostgreSQL RLS is expected to apply. The Supabase service-role key is backend-only and must never be exposed to the frontend or used for normal user-scoped queries in a way that bypasses RLS.

22. V1 Dashboard

23. Non-Functional Requirements

24. Edge Cases

25. Acceptance Criteria

26. Future Enhancements

27. V1 Development Phases

28. Recommended Project Structure

29. Data Lifecycle

30. Environment Variables (Appendix)

31. Recommended V1 Definition

Revision History

Revision	Summary	Changes
Rev 1	Initial draft	Original PRD: scope, roles, FRs, architecture, schema, API design.
Rev 2	Review pass	Locked open decisions (auth, question sourcing, verification, timezone, status source-of-truth); added error schema, env var appendix, updated diagrams, expanded phase table.
Rev 3	Navigation pass	Added static Table of Contents, revision history, and the question-deletion / soft-delete policy.
Rev 4	Implementation-readiness & branding pass	Applied Axly DSA Tracker branding (V1.0, dsatracker.axly.in); redesigned ER diagram for readability; simplified assignment status (removed "completed"); added full per-table DB specification; added Authentication vs Authorization flow; expanded API docs with auth/role/params/errors; clarified daily-question, assignment, and progress-calculation behavior; converted acceptance criteria to Given/When/Then; added edge cases, project structure, and data lifecycle sections; ran a full terminology consistency pass.
	Correctness & consistency pass	Replaced application-level duplicate-assignment checking with a database-level UNIQUE(user_id, question_id) constraint on assignments; added an explicit rule blocking deletion of the current daily question; defined the Assigned/Unassigned filter separately for User vs Admin context; clarified that Supabase Auth owns the Google OAuth flow and removed unused Google OAuth environment variables; tightened the progress-calculation rules (unassigned exclusion, historical retention, solved $\neq$ currently assigned); enlarged and relabeled the architecture and ER diagrams; expanded edge cases and acceptance criteria to match; ran a full terminology and cross-reference audit.

1. Executive Summary

Axly DSA Tracker (V1) is a role-based web application that helps users practice Data Structures and Algorithms consistently. The V1 release focuses on a simple daily-question workflow, difficulty and assignment filters, question assignment, self-reported progress tracking, and an administrative dashboard for managing questions and users.

2. Problem Statement

Learners often practice DSA without a structured routine and have difficulty tracking assigned questions, completion status, and progress by difficulty. Admins or mentors also need a simple way to assign questions and monitor participation.

3. V1 Objectives

- Provide secure authentication using Google OAuth.

- Support Admin and User roles with role-based access control (RBAC).
- Provide one global daily DSA question.
- Allow filtering by Easy, Medium, Hard and Assigned/Unassigned.
- Allow admins to create, update, soft-delete, assign and unassign questions manually.
- Track self-reported user question progress.
- Provide a clean dashboard for both users and administrators.

4. Scope

4.1 In Scope

- Authentication and session handling (Google OAuth)
- Role-based access control
- Question bank (admin-entered, manually managed)
- Daily question (UTC-based, single global question)
- Assignment, bulk assignment, and unassignment
- Difficulty and status filters with pagination
- Self-reported user progress tracker
- Admin dashboard
- Supabase/PostgreSQL database
- Versioned REST API (/api/v1)

4.2 Out of Scope for V1

- OTP-based authentication
- AI-generated questions or hints
- AI code review
- Competitive coding execution/judge
- Admin verification of submissions
- Advanced recommendation engine
- Leaderboard and badges
- Mobile application
- Complex analytics

5. V1 Locked Decisions

These decisions were resolved during review and are binding for V1 build scope. They are not to be re-opened without a documented technical reason.

Decision	Resolution	Rationale
Authentication method	Google OAuth only	Zero extra infra, fastest to ship, fits the target audience. OTP remains in Future Enhancements.
Question sourcing	Admin manual entry only	Matches lean-V1 scope. AI generation needs a review/QA workflow not built in V1.
Submission verification	Self-reported by user, no admin review	No code judge in V1; manual review does not scale. Matches habit-tracker positioning.
Daily question	One global question per UTC date, admin-selected, no auto-generation	Avoids per-user timezone complexity and keeps admin fully in control of content.
Daily question fallback	If no question is set for the date, the dashboard shows an explicit empty state, not an error	Prevents a broken dashboard when an admin forgets to schedule a question.
Status source of truth	assignments.status tracks assignment lifecycle only; submissions.status is the sole source of truth for solve progress	Removes ambiguity between two tables that previously overlapped in meaning.
Assignment status values	assigned, unassigned only — "completed" removed	"Completed" duplicated submissions.status and created two conflicting definitions of "done."
Question deletion policy	Soft delete only via is_active flag; never hard-deleted once assigned or submitted against	Preserves FK integrity and historical user progress.
Duplicate assignment prevention	Enforced by a database-level UNIQUE(user_id, question_id) constraint on assignments, not application-only logic	Guarantees correctness under concurrent requests; application logic alone cannot fully prevent race conditions.
Daily question deletion	An active question cannot be soft-deleted while it is the current daily question; admin must change or remove it first	Prevents the dashboard from referencing a deleted question and avoids a confirmation-flag workaround.

6. User Roles

Role	Permissions
User	Login, view daily question, browse/filter questions, view assigned questions, self-report progress, view own tracker.
Admin	All user capabilities plus manage questions, manage assignments (single/bulk), manage users/roles, view overall progress.

7. Core User Flow

- User opens the application.
- User authenticates using Google OAuth.
- Backend validates the authenticated session and role.
- User is redirected to the appropriate dashboard.
- The dashboard displays the daily question (or an empty state) and current tracker status.
- User filters questions by difficulty and assignment/status.
- User attempts a question and self-reports its status.
- The system stores progress and displays updated statistics.

8. Admin Flow

- Admin logs in and is verified through RBAC.
- Admin opens the question management area.
- Admin manually creates or edits DSA questions with difficulty and topic.
- Admin assigns or unassigns questions to selected users, individually or in bulk.
- Admin sets or changes the daily question for the current UTC date.
- Admin filters assignments and reviews user progress.

9. Functional Requirements

ID	Requirement	Priority
FR-01	User can authenticate with Google OAuth.	High
FR-02	System distinguishes Admin and User roles via backend-derived authorization.	High
FR-03	System displays one global daily question, selected on a UTC-day boundary.	High
FR-04	Questions can be filtered by Easy/Medium/Hard.	High
FR-05	Questions can be filtered by Assigned/Unassigned.	High
FR-06	Admin can create, edit and soft-delete questions manually.	High
FR-07	Admin can assign/unassign questions, individually or in bulk, with duplicate prevention.	High
FR-08	User can self-report question progress/status.	High
FR-09	User can view personal progress with computed completion percentage.	High
FR-10	Admin can view overall assignment/progress data.	Medium

ID	Requirement	Priority
FR-11	List endpoints (questions, assignments) support pagination and filtering.	Medium
FR-12	API responses follow a consistent error schema across endpoints.	Medium
FR-13	Admin can change the daily question for the current date; the change is reflected immediately.	Medium
FR-14	Admin cannot soft-delete a question that is the current daily question until it is changed or removed.	High

9.1 Assigned / Unassigned Filter Definition

The meaning of the Assigned/Unassigned filter depends on the requesting role, since "assigned" means something different from a user's perspective than from an admin's:

Role	Assigned means	Unassigned means
User	Questions currently assigned to the logged-in user (an assignments row with user_id = self and status = assigned).	Questions not currently assigned to the logged-in user.
Admin	Questions assigned to at least one user (at least one assignments row with status = assigned exists for the question).	Questions currently assigned to nobody (no assignments row with status = assigned exists for the question).

10. Recommended Technology Stack

Layer	Technology	Purpose
Frontend	React + Vite	Web UI and dashboard
Styling	Tailwind CSS	Responsive interface
Backend	Node.js + Express	Versioned REST API and business logic
Database	Supabase PostgreSQL	Relational data storage
Authentication	Supabase Auth / Google OAuth	Authentication and sessions
Authorization	Backend RBAC + PostgreSQL RLS	Access control
Version Control	Git + GitHub	Source control and collaboration
Testing	Playwright + API tests	E2E and API validation

11. System Architecture

The architecture separates presentation, API/business logic, authentication, and persistent data. Authentication is scoped to Google OAuth for V1; the API is versioned under /api/v1.

DSA Tracker V1 — High-Level Architecture

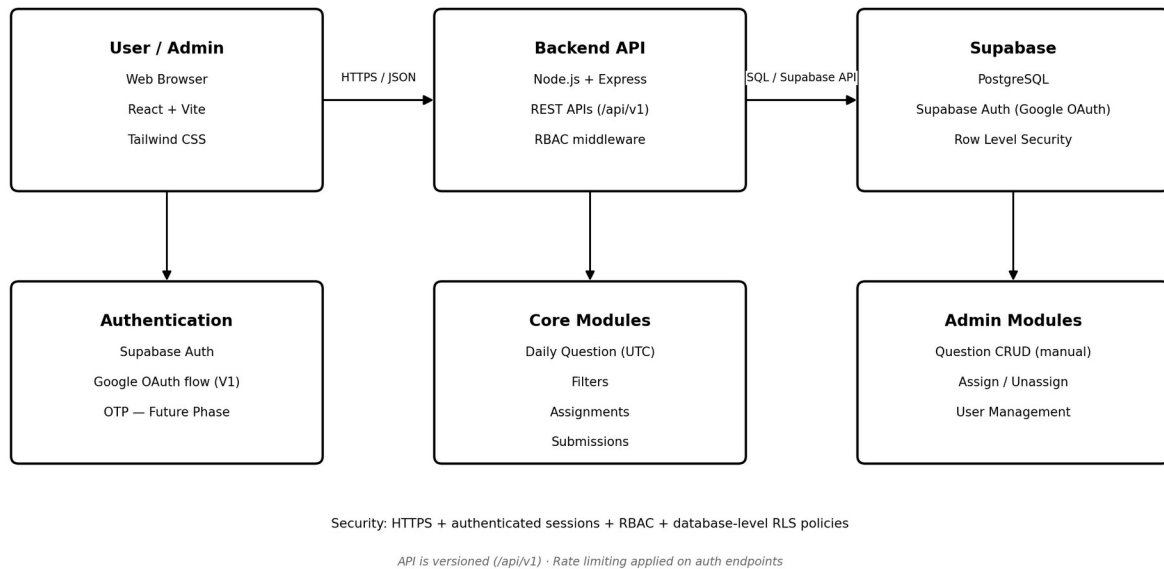


Figure 1 — High-Level Architecture

12. Architecture Components

Component	Responsibilities
React Frontend	Login screens, dashboards, filters, question views, tracker and admin UI. Never determines role — only reflects what the backend returns.
Express API	Validation, RBAC middleware, question/assignment/progress APIs, rate limiting on auth routes, business rules.
Supabase Auth	Google OAuth flow and session management.
PostgreSQL	Users, questions, topics, assignments, daily questions and submissions.
RLS	Database-level restrictions so users access only permitted data, independent of API-layer bugs.

13. Authentication & Authorization

Authentication and authorization are distinct concerns and must not be conflated in implementation:

- Authentication answers "who is this user?" — handled by Google OAuth and Supabase Auth.
- Authorization answers "what can this user do?" — handled by backend RBAC middleware and PostgreSQL RLS.

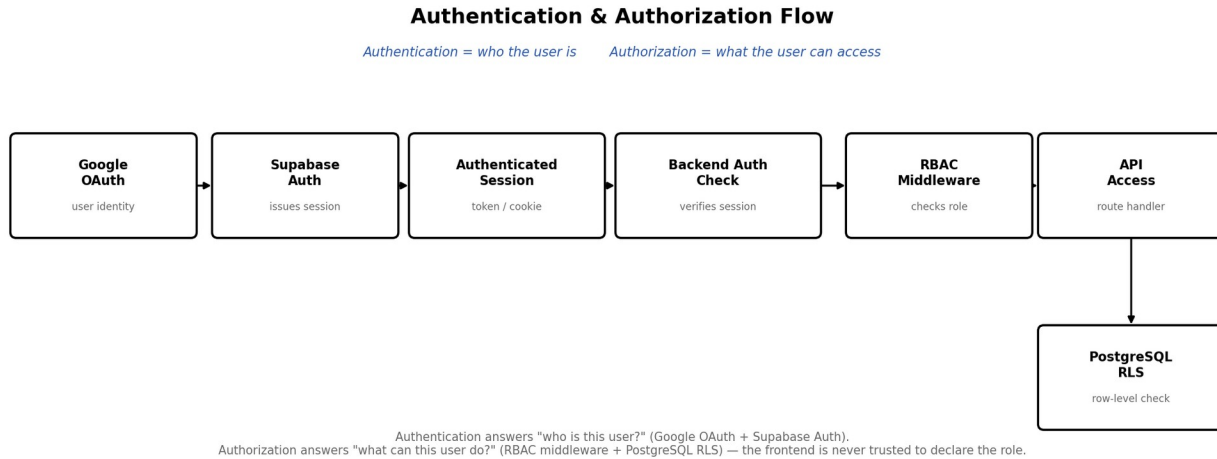


Figure 2 — Authentication & Authorization Flow

Flow: Google OAuth → Supabase Auth → authenticated session → backend authentication check → RBAC middleware → API access → PostgreSQL RLS.

13.1 Supabase Auth Owns the OAuth Flow

- Supabase Auth handles the Google OAuth authentication flow end to end — the redirect, token exchange, and session issuance.
- Google OAuth credentials (client ID and client secret) are configured directly in the Supabase Auth provider settings, not as separate application-level secrets.
- The Google client secret is held by Supabase and must never be exposed to, or duplicated in, the frontend.
- The frontend must not implement a second, independent Google OAuth flow — it only triggers the Supabase Auth sign-in call and receives the resulting session.
- Backend authorization is derived from the authenticated Supabase session on every request, never from any role or identity claim sent by the frontend.

The frontend must never be trusted to determine or transmit the user's role for authorization purposes. Role is always re-derived server-side from the authenticated session on every request.

14. Entity Relationship Diagram

Logical structure: users → assignments → questions → submissions, with questions → topics, questions → daily_questions, and users → roles.

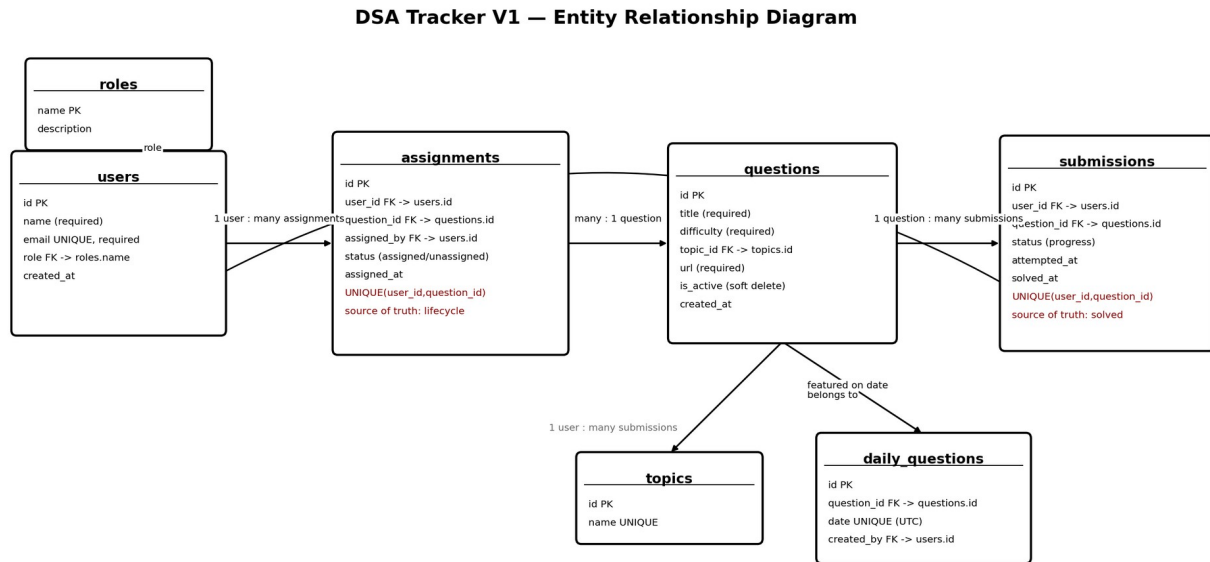


Figure 3 — Entity Relationship Diagram (full field-level spec in Section 15)

15. Database Table Specifications

For every table: primary key, foreign keys, required vs optional fields, unique constraints, indexes, soft-delete behavior, and timestamps.

15.1 users

Attribute	Detail
Primary key	id
Foreign keys	role -> roles.name
Required fields	name, email, role
Unique constraints	email
Indexes	email (unique index), role (lookup)
Soft delete	Not applicable in V1 — users are not deleted, only deactivated at auth-provider level if needed

Attribute	Detail
Timestamps	created_at

15.2 roles

Attribute	Detail
Primary key	name
Foreign keys	None
Required fields	name
Unique constraints	name (as primary key)
Indexes	Primary key index only
Soft delete	Not applicable — fixed set (admin, user)
Timestamps	Not required

15.3 questions

Attribute	Detail
Primary key	id
Foreign keys	topic_id -> topics.id
Required fields	title, difficulty, url
Optional fields	topic_id
Unique constraints	None required; duplicate-title check enforced at application layer on create
Indexes	difficulty, topic_id, is_active (all frequently filtered)
Soft delete	is_active boolean, default true. Set to false on delete — see Section 15.8 for FK behavior.
Timestamps	created_at

15.4 topics

Attribute	Detail
Primary key	id
Foreign keys	None
Required fields	name

Attribute	Detail
Unique constraints	name
Indexes	name (unique index)
Soft delete	Not applicable in V1
Timestamps	Not required

15.5 assignments

Attribute	Detail
Primary key	id
Foreign keys	user_id -> users.id, question_id -> questions.id, assigned_by -> users.id
Required fields	user_id, question_id, assigned_by, status
Unique constraints	UNIQUE(user_id, question_id) — at most one assignment row per user per question, enforced by the database, not application logic
Indexes	user_id, question_id, status (all frequently filtered); the UNIQUE constraint also serves as an index
Soft delete	Not applicable — unassignment changes status, does not delete the row (Section 18)
Timestamps	assigned_at

15.6 submissions

Attribute	Detail
Primary key	id
Foreign keys	user_id -> users.id, question_id -> questions.id
Required fields	user_id, question_id, status
Optional fields	attempted_at, solved_at
Unique constraints	UNIQUE(user_id, question_id) — one submission row per user per question
Indexes	user_id, question_id, status
Soft delete	Not applicable — submissions persist even if the related assignment is removed (Section 18)
Timestamps	attempted_at, solved_at (both nullable until reached)

15.7 daily_questions

Attribute	Detail
Primary key	id
Foreign keys	question_id -> questions.id, created_by -> users.id
Required fields	question_id, date, created_by
Unique constraints	date — exactly one daily question per UTC date
Indexes	date (unique index)
Soft delete	Not applicable
Timestamps	created_at implicit via row creation; date is the UTC calendar date the question applies to

15.8 Foreign Key Behavior on Soft Delete

When a question is soft-deleted (is_active set to false):

- Existing rows in assignments and submissions that reference the question are never deleted or nulled — history remains intact.
- The question no longer appears in active question lists or filters (GET /api/v1/questions excludes is_active = false by default).
- Admin cannot create a new assignment against an inactive question; the API returns a validation error (Section 24).
- An active question cannot be soft-deleted while it is the current daily question — see Section 17 for the exact rule and error behavior.

16. Status Value Definitions

Two independent status fields exist by design and must not be treated as interchangeable:

Field	Values	Meaning	Source of truth for
assignments.status	assigned, unassigned	Whether a question is currently assigned to a user	Assignment lifecycle only
submissions.status	not_started, attempted, solved, skipped	The user's self-reported solving progress	"Solved" — the only place solve state is determined

"Completed" was removed from assignments.status in Revision 4 because it duplicated submissions.status = solved and created two conflicting definitions of done. An assignment is either assigned or unassigned; whether the underlying question has been solved is answered exclusively by the linked submission row.

17. Daily Question Behavior

- Exactly one `daily_questions` row exists per UTC calendar date (enforced by the unique constraint on date).
- The question is selected manually by an admin — there is no automatic generation or rotation.
- `GET /api/v1/daily-question` resolves "today" using the UTC date at request time.
- If no row exists for the current UTC date, the endpoint returns a defined empty-state response (not an error); the frontend renders "No daily question set for today."
- If an admin changes the daily question for the current date, the existing row for that date is updated in place (same date, new `question_id`) — it does not create a second row for the same date, and the change is reflected on the next dashboard load with no caching delay.

17.1 Deleting the Current Daily Question

V1 rule: an active question cannot be soft-deleted while it is the current daily question. The admin must first change or remove the daily question before the question itself can be deleted.

- If an admin calls `DELETE /api/v1/questions/:id` for a question that is referenced by today's `daily_questions` row, the API rejects the request with 409 Conflict and an error message identifying the conflict ("Cannot delete the current daily question — change it first").
- No confirmation flag or override exists in V1; the admin's only path is to set a different daily question (or explicitly clear it) and then retry the delete.
- Once the daily question for the current date has been changed to a different question (or removed), the previously blocked question becomes deletable through the normal soft-delete flow.
- This rule applies only to today's UTC date. A question referenced only by a past `daily_questions` row (a previous date) is not blocked from deletion — soft delete proceeds normally, and the historical `daily_questions` row is retained unchanged.
- The existing empty-state behavior is unaffected: if no daily question is set at all, the dashboard shows the empty state described above, independent of this deletion rule.

18. Assignment & Unassignment Behavior

18.1 Single and Bulk Assignment

- `POST /api/v1/assignments` creates one assignment for one user/question pair.
- `POST /api/v1/assignments/bulk` creates assignments for multiple users against one or more questions in a single request.

18.2 Duplicate Assignment Prevention

Duplicate assignments are prevented at the database layer by a `UNIQUE(user_id, question_id)` constraint on assignments — not by an application-level check alone. This guarantees correctness even under concurrent requests.

- On `POST /api/v1/assignments`, the backend attempts an insert (or upsert) against the assignments table. If a row for that `user_id/question_id` pair already exists, the database constraint prevents a second row from being created.

- If the existing row has status = assigned, the API returns 409 Conflict — the question is already assigned to that user.
- If the existing row has status = unassigned, the API performs an update (not an insert) that reuses the existing row and sets status back to assigned — see Section 18.3.
- Because the constraint is enforced by the database, two concurrent requests attempting to assign the same user/question pair for the first time cannot both succeed in creating separate rows: the database guarantees only one insert wins, and the second request either receives a 409 or is resolved as a reassignment, never a duplicate row.

18.3 Re-assigning an Unassigned Question

If a question was previously unassigned to a user (status = unassigned) and is assigned again, the existing assignment row — located via the `UNIQUE(user_id, question_id)` constraint — is reused and its status is updated back to assigned, rather than inserting a new row. `assigned_at` is updated to the new assignment time.

18.4 Unassignment

`DELETE /api/v1/assignments/:id` does not delete the row. It sets status = unassigned. This preserves assignment history (who assigned what, when) for admin reporting.

18.5 Submission History After Unassignment

If a user has already submitted progress (submissions row) for a question that is later unassigned, the submissions row is never deleted or modified by the unassignment. A user's solved/attempted history is permanent and independent of current assignment state.

19. Progress & Completion Model

The dashboard derives all progress metrics from submissions and assignments as follows, with `submissions.status` as the exclusive source of truth for "solved":

Metric	Calculation
Assigned questions	<code>COUNT(assignments WHERE user_id = :user AND status = 'assigned')</code>
Attempted questions	<code>COUNT(submissions WHERE user_id = :user AND status IN ('attempted','solved') AND question is currently assigned)</code>
Solved questions	<code>COUNT(submissions WHERE user_id = :user AND status = 'solved' AND question is currently assigned)</code>
Completion percentage	<code>Solved currently-assigned active questions ÷ Currently-assigned active questions × 100</code>

Because `assignments.status` never encodes solve state, there is exactly one place a developer can look to determine whether a question is solved: `submissions.status`.

19.1 Rules Governing the Calculation

- The completion percentage denominator includes only questions with an assignments row where status = assigned and the question's is_active = true. Unassigned questions are excluded from the current completion denominator, even if a submission exists for them.
- Historical submissions are never deleted when an assignment is unassigned. The submissions row persists exactly as it was.
- Historical submission data remains available for history/audit purposes (e.g. an admin reviewing what a user solved in the past), even though it no longer counts toward the current completion percentage once the assignment is removed.
- A solved submission does not automatically mean the question is currently assigned — a user may have solved a question that was later unassigned; submissions.status = solved remains true, but the question drops out of the completion denominator.
- Assignment state (assignments.status) and solving state (submissions.status) remain fully separate at all times; neither field is derived from or overwritten by the other.

20. API Design

All endpoints are versioned under `/api/v1`. Every list endpoint supports pagination via `?page` and `?limit` query parameters (default limit 20, max 100) and returns `{ data, page, limit, total }`.

20.1 Authentication

Method	Endpoint	Auth	Role	Purpose
POST	<code>/api/v1/auth/verify</code>	Session required	Any authenticated	Verifies the current session is valid and returns the resolved role.

20.2 Questions

Method	Endpoint	Auth	Role	Purpose
GET	<code>/api/v1/questions</code>	Required	User/Admin	List/filter active questions. Params: difficulty, topic_id, assigned (true/false — meaning depends on role, see Section 9.1), page, limit.
POST	<code>/api/v1/questions</code>	Required	Admin	Create a question. Body: title, difficulty, topic_id, url.
PATCH	<code>/api/v1/questions/:id</code>	Required	Admin	Update a question's fields.
DELETE	<code>/api/v1/questions/:id</code>	Required	Admin	Soft-delete: sets <code>is_active = false</code> . Returns 409 if the question is the current daily question (Section 17.1).

Common errors: 400 (invalid difficulty value), 404 (question not found), 409 (duplicate title on create, or question is the current daily question and must be changed before deletion).

20.3 Daily Question

Method	Endpoint	Auth	Role	Purpose
GET	<code>/api/v1/daily-question</code>	Required	User/Admin	Returns today's (UTC) daily question, or an empty-state payload if none is set.

Method	Endpoint	Auth	Role	Purpose
POST	/api/v1/daily-question	Required	Admin	Sets or replaces the daily question for the current UTC date. Body: question_id.

Common errors: 400 (question_id references an inactive question), 404 (question not found).

20.4 Assignments

Method	Endpoint	Auth	Role	Purpose
POST	/api/v1/assignments	Required	Admin	Assign one question to one user. Body: user_id, question_id. Relies on the UNIQUE(user_id, question_id) database constraint — reuses the existing row if one exists (Section 18.2/18.3).
POST	/api/v1/assignments/bulk	Required	Admin	Assign question(s) to multiple users. Body: user_ids[], question_ids[]. Each pair is subject to the same UNIQUE constraint individually.
DELETE	/api/v1/assignments/:id	Required	Admin	Unassign — sets status = unassigned, row is retained.
GET	/api/v1/assignments	Required	User/Admin	List/filter assignments. Params: user_id (admin only), status, page, limit.

Common errors: 404 (user or question not found), 409 (question already assigned to that user — existing row has status = assigned), 400 (question is inactive).

20.5 Submissions & Progress

Method	Endpoint	Auth	Role	Purpose
PATCH	/api/v1/submissions/:id	Required	User (own record only)	Update self-reported status. Body: status.
GET	/api/v1/progress/me	Required	User	Returns own assigned/attempted/solved counts and completion percentage.
GET	/api/v1/admin/progress	Required	Admin	Returns aggregate progress across all users. Params: page,

Method	Endpoint	Auth	Role	Purpose
				limit.

Common errors: 403 (user attempting to update another user's submission), 400 (invalid status value).

20.6 Standard Error Response Shape

```
{ "error": { "code": "VALIDATION_ERROR", "message": "difficulty must be one of easy | medium | hard", "field": "difficulty" } }
```

21. Security Requirements

- Google OAuth is the sole identity provider for V1; no password storage.
- Every request re-validates the Supabase session server-side — the frontend cannot assert identity or role.
- RBAC middleware enforces role checks on every admin-only route before the handler executes.
- PostgreSQL Row Level Security policies enforce access at the database layer as a second line of defense, independent of API-layer bugs.
- All request bodies and query parameters are validated on the backend (type, range, enum membership).
- Rate limiting is applied to authentication endpoints to reduce abuse risk.
- CORS is restricted to known frontend origins; wildcard origins are not permitted in production.
- HTTPS is enforced in production for all traffic.
- Admin actions (question create/edit/delete, assign/unassign, daily-question changes) are logged with actor, action, and timestamp for traceability.
- A user can only read or update their own submissions rows; ownership is checked server-side on every request, never inferred from client input.
- Soft-deleted questions (is_active = false) cannot be viewed, assigned, or modified through normal user-facing APIs — they are excluded from user-facing GET/PATCH/assignment paths. Admin-facing endpoints continue to reference them only for historical reporting (Section 15.8).

21.1 Secret Management

Google OAuth client credentials are held entirely within Supabase Auth's provider configuration and are never part of the application's own environment variables or codebase (Section 13.1).

Secret	Frontend-safe?	Notes
SUPABASE_URL	Yes	Public project URL
SUPABASE_ANON_KEY	Yes	Public client key, restricted by RLS
SUPABASE_SERVICE_ROLE_KEY	No — backend only	Bypasses RLS; must never reach the frontend bundle

22. V1 Dashboard

User Dashboard

- Today's question (or empty state)
- Questions solved
- Pending assignments
- Difficulty-wise progress
- Recent activity
- Current completion percentage

Admin Dashboard

- Total users
- Total active questions
- Assignments
- Solved vs pending (from submissions.status)
- Question management
- User management

23. Non-Functional Requirements

- Responsive on desktop and mobile browsers.
- Authenticated pages must not be accessible without a valid session.
- Admin-only actions must be blocked at both API and database layers.
- API responses use consistent JSON structures and HTTP status codes, per the error schema in Section 20.6.
- Database queries use indexes for frequently filtered fields such as difficulty, user_id, question_id, status, and date.
- List endpoints are paginated to remain performant as data grows.
- The application is deployable independently as frontend and backend services.

24. Edge Cases

Case	Expected Behavior
No daily question exists for today	Dashboard shows an explicit empty state, not an error.
Today's daily question is inactive	Cannot occur under normal operation — Section 17.1 blocks deleting a question while it is the current daily question. If encountered, treated as the empty-state case.
Admin tries to soft-delete today's	API returns 409 Conflict; admin must change or remove the daily question

Case	Expected Behavior
daily question	first (Section 17.1).
Duplicate assignment (already assigned)	API returns 409 Conflict; the UNIQUE(user_id, question_id) constraint prevents a duplicate row.
Concurrent duplicate assignment requests	The database UNIQUE constraint allows only one row to exist; the losing concurrent request receives 409 Conflict or is resolved as a reassignment, never a duplicate row.
Reassignment after unassignment	The existing assignment row (matched by the UNIQUE constraint) is reused and its status is set back to assigned; no new row is created.
Duplicate submission update for the same question	Prevented by UNIQUE(user_id, question_id) on submissions; PATCH updates the existing row rather than creating a second one.
Invalid difficulty value submitted	API returns 400 with a field-level validation error.
Unauthorized access to an admin endpoint	API returns 403 Forbidden; RBAC middleware blocks before the handler runs.
User attempts to update another user's submission	API returns 403 Forbidden; ownership is checked server-side.
Assignment removed after a submission exists	Assignment status becomes unassigned; the submission row and its history remain untouched, but the question drops out of the current completion denominator (Section 19.1).
Pagination requested beyond available records	API returns 200 with an empty data array, not an error.

25. Acceptance Criteria

Written as testable Given/When/Then statements.

25.1 Authentication

- Given a new user, when they sign in with Google OAuth, then a session is created and a user record exists with a default role.
- Given an authenticated session, when it expires or is invalid, then protected routes return 401 and redirect to login.

25.2 RBAC

- Given an authenticated user with role "user", when they call an admin-only endpoint, then the API returns 403 Forbidden.
- Given an authenticated admin, when they call any admin endpoint, then the request succeeds subject to normal validation.

25.3 Question CRUD & Soft Delete

- Given an authenticated admin, when they create a question with a valid difficulty and required fields, then the question appears in the active question list.
- Given a question with existing assignments or submissions, when an admin deletes it, then `is_active` is set to false and no assignment or submission rows are removed.

25.4 Daily Question

- Given an admin sets a question as today's daily question, when a user requests `/api/v1/daily-question`, then that question is returned for the current UTC date.
- Given no daily question is set for the current UTC date, when a user requests the daily question, then the API returns the defined empty-state response.
- Given a question is the current daily question, when an admin attempts to soft-delete it, then the API returns 409 Conflict and the question remains active.
- Given the daily question has been changed to a different question, when an admin then soft-deletes the previously-blocked question, then the delete succeeds and `is_active` is set to false.

25.5 Difficulty & Assigned/Unassigned Filtering

- Given a set of questions with mixed difficulties, when a user filters by `difficulty=medium`, then only medium questions are returned, paginated per the requested `page/limit`.
- Given a user filters by `assigned=true`, when the request is processed, then only questions with an active assignment to that specific user are returned.
- Given an admin filters by `assigned=false`, when the request is processed, then only questions with no user currently assigned to them are returned.

25.6 Assignment & Bulk Assignment

- Given an authenticated admin, when the admin assigns an active question to a user, then the system creates an assignment with status assigned and the question appears in that user's assigned-question list.
- Given an authenticated admin, when the admin bulk-assigns a question to multiple users, then an assignment is created for each user without violating the UNIQUE(user_id, question_id) constraint for any pair.

25.7 Unassignment

- Given an existing assignment, when an admin unassigns it, then its status becomes unassigned, the row is retained, and any related submission history is unchanged.

25.7a Duplicate Assignment Prevention

- Given a user already has an active assignment for a question, when an admin attempts to assign that same question to that same user again, then the API returns 409 Conflict and no duplicate row is created.
- Given two concurrent requests attempt to assign the same user/question pair for the first time, when both are processed, then the database UNIQUE constraint ensures only one assignment row exists afterward.

25.8 Submission Status

- Given an assigned question, when a user marks it as solved, then submissions.status is updated to solved and this is the only field the dashboard reads to determine solved state.
- Given a user's own submission, when another user attempts to update it, then the API returns 403 Forbidden.

25.9 Progress Calculation

- Given a user with 10 assigned questions and 4 with submissions.status = solved, when the user views their tracker, then completion percentage displays as 40%.
- Given a user solved a question and it is later unassigned, when the user views their tracker, then that question is excluded from the completion denominator, but its submissions row still shows status = solved when viewed in history.

25.10 API Errors & RLS

- Given an invalid request body, when submitted to any write endpoint, then the API returns 400 with the standard error schema identifying the offending field.
- Given RLS policies are active, when a direct database query is attempted outside an authenticated context, then no rows are returned.

26. Future Enhancements

- OTP-based authentication
- AI-assisted question generation (with admin review queue)

- Streaks and gamification
- Leaderboard
- Topic-wise learning roadmap
- Personalized question recommendations
- AI hints and explanations
- Code submission and automated judging (enables real solution verification)
- Advanced analytics
- Email/push reminders
- Mobile app

27. V1 Development Phases

Phase	Work
Phase 0	Decisions lock: confirm auth method, question sourcing, verification model, daily-question rule, assignment status values. No re-litigation once Phase 1 starts.
Phase 1	Project setup, repo structure, Supabase project init, environment variable configuration, database schema and constraints.
Phase 2	Google OAuth integration, session handling, RBAC middleware, RLS policies.
Phase 3	Question bank: admin manual CRUD, soft-delete, duplicate-title check, difficulty/topic filtering, pagination.
Phase 4	Daily question logic (UTC selection, empty-state fallback, admin change-in-place, delete-block while current), assignment system, bulk-assign, UNIQUE(user_id, question_id) constraint for duplicate-assignment prevention.
Phase 5	User tracker (self-reported status), progress calculation, admin dashboard, pagination on all list views.
Phase 6	Testing (Playwright: auth flow, RBAC boundaries, filter correctness; API validation tests; RLS policy tests), security review, independent frontend/backend deployment.

28. Recommended Project Structure

```

frontend/
src/
  components/    shared UI (buttons, cards, filters, modals)
  pages/         route-level views (Login, UserDashboard, AdminDashboard, Questions)
  services/      API client wrappers (questions.js, assignments.js, submissions.js)
  hooks/         data-fetching and auth hooks
  context/       auth/session context
public/

backend/
src/
  routes/        /api/v1 route definitions per resource
  controllers/   request handling, calls services
  services/      business logic (assignment rules, progress calc)
  middleware/    auth check, RBAC, rate limiting, error handler
  validation/    request schema validation (per-endpoint)
  db/            Supabase client, query helpers

database/
migrations/     schema and constraint definitions
policies/       RLS policy definitions per table

docs/
PRD.docx        this document
api-reference.md

tests/
e2e/            Playwright specs
api/            API contract/integration tests

```

29. Data Lifecycle

End-to-end lifecycle of a question and its dependent records:

- 1. Question creation — admin manually enters a question (title, difficulty, topic, url); `is_active` defaults to true.
- 2. Assignment — admin assigns the question to one or more users, creating assignments rows.
- 3. User submission — a user self-reports progress, creating a submissions row (UNIQUE per user/question).
- 4. Question soft deletion — admin sets `is_active = false`; the question is hidden from active lists and blocked from new assignments.
- 5. Historical data retention — existing assignments and submissions rows referencing the question are never deleted, updated, or nulled as a result of the soft delete. A user's progress history remains permanently accurate even after the underlying question is retired.

This ordering guarantees that no historical record of a user's work can be silently lost by routine question-bank maintenance.

30. Environment Variables (Appendix)

Only the variables actually required by the V1 architecture are listed. Google OAuth client credentials are configured directly in the Supabase Auth provider settings (Section 13.1), not as separate application environment variables.

Variable	Used By	Notes
SUPABASE_URL	Backend, Frontend	Project URL
SUPABASE_ANON_KEY	Frontend	Public client key, restricted by RLS
SUPABASE_SERVICE_ROLE_KEY	Backend only	Bypasses RLS; never exposed to frontend
API_BASE_URL	Frontend	Points to /api/v1

31. Recommended V1 Definition

The V1 should be considered complete when a user can securely log in via Google OAuth, receive a daily DSA question, filter and view questions, complete assigned questions with self-reported status, and track progress, while an admin can manually manage the question bank, assignments, daily question, and user access. Advanced AI, leaderboard, recommendation, and submission-verification features remain outside V1.